A Serverless Fog Computing Architecture for Smart Waste Management: Design, Deployment, and Evaluation on AWS

Gundeti Sachin Reddy
Student ID: X23432721
MSc in Cloud Computing
National College of Ireland
Dublin, Ireland
x23432721@student.ncirl.ie

***Abstract*—This paper presents the design, implementation, and real-world cloud deployment of a fog and edge computing pipeline for smart municipal waste management. The system simulates five distinct sensor types (bin fill level, internal temperature, gas concentration, bin weight, and lid-open events) across two collection districts, each independently configurable for sampling and dispatch frequency. A Node.js fog node performs windowed aggregation and threshold-based alerting before dispatching processed batches to a scalable cloud backend built from Amazon SQS, AWS Lambda, and Amazon DynamoDB. The dashboard, originally hosted on a persistent EC2 instance, was subsequently migrated to a fully serverless architecture (Amazon S3, AWS Lambda, and Amazon API Gateway) to align the entire backend with the module's function-as-a-service scalability criterion and to resolve a genuine campus-network access restriction. The complete system was deployed and tested on a real AWS account rather than a local emulator alone, which surfaced defects invisible to local testing, including credential-handling bugs that only manifest under real temporary AWS credentials, and undocumented platform restrictions specific to the AWS Academy Learner Lab environment. This paper critically analyses the architectural decisions taken, the defects discovered during real deployment, and the lessons learned for building genuinely cloud-native fog computing systems.**

Keywords—fog computing, edge computing, Internet of Things, serverless architecture, AWS Lambda, cloud deployment, smart waste management

I. Introduction

Municipal waste collection is traditionally scheduled on a fixed calendar rather than on actual bin conditions, which routinely results in either premature collection runs to bins that are not yet full or, conversely, overflowing bins and associated fire and odour hazards that are only discovered after the fact. Beyond the direct cost of wasted collection runs, a fixed schedule also leaves genuinely hazardous conditions, an overheating bin, a gas build-up, evidence of tampering, undetected between scheduled visits, precisely the conditions a condition-driven system is meant to surface immediately rather than on the next calendar date. Internet of Things sensing, combined with fog and edge processing close to the data source, allows a waste authority to move from a fixed schedule to a condition-driven one, collecting the busiest bins first while still processing data efficiently rather than streaming every raw reading to the cloud.

The National Institute of Standards and Technology defines cloud computing around five essential characteristics, on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service [9]; fog computing, in turn, is not a rejection of that model but an extension of it. Bonomi et al. define fog computing as a platform that extends cloud services to the edge of the network, characterised by low latency, geographic distribution, and support for a large number of nodes [5]. This project adopts that definition directly: aggregation and alerting are pushed to a fog node co-located with the sensors, and only reduced, already-meaningful data crosses the network boundary into the cloud proper, rather than a "thin edge, thick cloud" design in which every raw reading is streamed upstream for processing. The cloud-side design decisions in Section II, in turn, are justified against the AWS Well-Architected Framework's operational-excellence, reliability, and cost-optimisation pillars [10], rather than by convention alone.

This paper documents the design and construction of a fog and edge computing solution for this domain, built as an individual assignment for the Fog and Edge Computing (H9FECC) module. The brief for this coursework required, at minimum: (1) a sensor and fog layer generating data from three to five distinct sensor types with independently configurable sampling and dispatch rates, where the fog node genuinely receives and processes sensor data before dispatching it onward; (2) a scalable backend layer, designed around mechanisms such as queues, function-as-a-service (FaaS), or autoscaling, that processes the fog node's output and exposes a responsive dashboard for the sensor types; and (3) deployment and testing of that backend on a public cloud platform, not merely a local simulation.

The remainder of this paper is organised as follows. Section II describes the system's architecture and the design decisions taken at each layer, including a mid-project migration of the dashboard to a fully serverless model, a critical comparison against the architectural alternatives that were not chosen, and the security posture of the resulting deployment. Section III describes the concrete implementation: the software components and libraries used, the automated test suite, the continuous integration and deployment pipeline, and the real AWS deployment together with the genuine defects it surfaced. Section IV evaluates the deployed system against measured evidence rather than intent alone. Section V concludes with a critical reflection on the project and directions for future work.

II. System Architecture and Design

A. Sensor and Fog Layer

The simulated deployment models two waste-collection districts, district-a and district-b, each instrumented with five sensor types: fill_level_pct, internal_temp_c, gas_level_ppm, bin_weight_kg, and lid_open_count. Every one of the ten resulting sensor processes (five types across two districts) exposes two independent, separately configurable environment variables, SAMPLE_INTERVAL and DISPATCH_INTERVAL, satisfying the brief's requirement that generation frequency and dispatch rate be genuinely decoupled knobs rather than a single aliased value. Each sensor process was verified to hold a pairwise-distinct combination of these two values, since a duplicated pair would otherwise silently fail to demonstrate that the two knobs are truly independent.

The fog node is a Node.js HTTP service that ingests raw readings over its own REST endpoint, buffers them per sensor type and site, and performs genuine windowed aggregation, not a pass-through relay, computing a minimum, maximum, average, latest value, and reading count over a fixed time window. Each aggregated window is evaluated against the domain-specific threshold rules listed in Table I, and the resulting aggregate-plus-alerts payload is dispatched as a single message to an Amazon SQS queue. Performing aggregation and alerting in the fog layer, rather than forwarding every raw reading to the cloud, is the central architectural argument for calling this layer "fog" rather than a simple network relay: it reduces the volume and frequency of cloud-bound traffic while still preserving the information a backend consumer needs.

TABLE I. Fog Layer Alert Rules

Sensor Type	Rule	Alert Key
fill_level_pct	avg > 85%	collection_needed
internal_temp_c	avg > 55°C	fire_risk_warning
gas_level_ppm	avg > 400 ppm	odor_gas_exceedance
lid_open_count	max > 8	tamper_suspected
bin_weight_kg	(no rule)	—

bin_weight_kg carries no threshold rule deliberately: weight alone does not indicate a collection-worthy or hazardous state independently of fill level, so no alert was defined for it, rather than an arbitrary one being added purely for symmetry with the other four sensor types.

B. Scalable Backend Layer

The backend layer is built entirely from managed, scalable AWS primitives rather than a hand-rolled server: an Amazon SQS [3] standard queue decouples the fog node's dispatch rate from the backend's processing rate, and an AWS Lambda [1] function, subscribed to that queue via an event-source mapping, is invoked automatically as messages arrive, writing each aggregated window into an Amazon DynamoDB [2] table. This queue-plus-FaaS pairing is one of the scalability mechanisms the brief explicitly names as an example, and it was chosen specifically because it requires no server capacity planning: the queue absorbs bursts, and Lambda's own concurrency model scales the number of concurrent invocations to the backlog without any manual provisioning.

A commonly raised critique of Lambda-based architectures is cold-start latency: an invocation on a function with no currently warm execution environment incurs an additional initialisation delay before the handler itself runs. This is a genuine cost of the FaaS approach, but is judged acceptable here for two reasons specific to this workload. First, the fog-to-processor path is asynchronous (SQS-triggered), not a user-facing request/response path, so an occasional few-hundred-millisecond cold start delays when a reading is durably stored, not a user-visible page load. Second, the dashboard-facing Lambda function is invoked frequently enough by the live dashboard's own polling that its execution environment rarely goes cold during an active demonstration session. Cold-start latency would be a materially more serious concern for a synchronous, user-facing request on a rarely invoked function, which is not the shape of either Lambda function in this system.

The DynamoDB table uses sensor_type as its partition key and a composite sort key of the form window_end#site_id as its range key. This design choice is deliberate: if the sort key were window_end alone, two districts producing an aggregate for the same sensor type in the same flush cycle would collide on an identical primary key, and whichever write landed second would silently overwrite the first, permanently losing one district's reading for that window. Appending the site identifier to the sort key disambiguates the two districts' concurrent writes without requiring any additional index, and was verified directly by confirming that both districts' aggregates for a shared window are retained independently rather than one clobbering the other. The table itself runs in on-demand (pay-per-request) capacity mode rather than provisioned throughput, which removes the need to forecast a read/write capacity plan for a workload whose volume is not known in advance, at the cost of a marginally higher per-request price than well-forecast provisioned capacity would achieve; at this project's data volume the difference is immaterial, and the operational simplicity of not managing capacity was judged the better trade for a project of this scale.

C. Dashboard Layer and Its Migration to a Fully Serverless Architecture

The dashboard was originally implemented as a fourth long-running container alongside the fog node and the ten sensor processes, exposing a small REST API and a static single-page frontend from one Node.js HTTP server on a dedicated port of the same EC2 instance used for the fog node. This was a pragmatic first deployment, but on critical review it had two weaknesses. First, it was the one piece of the "scalable backend" requirement still resting on a persistent server rather than on a queue or FaaS mechanism, unlike every other backend component. Second, and more concretely, a raw EC2 public IP address on a non-standard port served over plain HTTP is exactly the traffic shape that campus and corporate WiFi networks commonly block by policy; this was confirmed as a genuine, reported access failure rather than a hypothetical concern.

The dashboard was therefore migrated to a fully serverless model: its static assets (HTML, CSS, and JavaScript) are now served directly from an Amazon S3 bucket over the bucket's own HTTPS endpoint, and its small REST API was extracted into its own AWS Lambda function, reusing the existing route-handling logic unchanged through a thin adapter that translates between the API gateway's event format and the original Node.js request/response shapes. Two platform-specific constraints of the AWS Academy Learner Lab account used for this deployment were discovered empirically during this migration and are worth recording as design evidence: Amazon CloudFront is unavailable in this account, including read-only operations, so the S3 bucket is served over its own virtual-hosted-style HTTPS endpoint rather than through a CloudFront distribution; and public, unauthenticated AWS Lambda Function URLs are also

blocked, confirmed by a successful authenticated invocation of the same function returning a Forbidden response when invoked through its Function URL. Amazon API Gateway [4] was not subject to either restriction and was used instead as the public entry point in front of the dashboard's Lambda function.

The fog node and the ten sensor processes remain on the EC2 instance. This was a deliberate, and not merely residual, architectural decision: both are continuous, long-running processes rather than naturally invoke-on-demand units of work, which makes them a poor fit for Lambda's request-driven execution model, and neither is ever reached directly by a browser, so the campus-network restriction that motivated moving the dashboard does not apply to them.

D. Deployment Topology

Fig. 1 summarises the resulting topology. Sensor processes and the fog node run in Docker containers on a single EC2 instance, managed entirely through AWS Systems Manager Session Manager rather than SSH, so no key pair is provisioned and no inbound SSH port is opened on the instance's security group. The fog node dispatches aggregated windows to an SQS queue, which triggers a Lambda function that writes to a DynamoDB table. A second Lambda function, sitting behind an API Gateway HTTP API, reads from that same table (and from the queue and the first Lambda function's own metadata) to answer the dashboard's REST API calls, while the dashboard's static frontend is served independently from Amazon S3, reached directly by the browser. This topology places every backend component except the sensor and fog tier behind a managed, serverless service boundary.

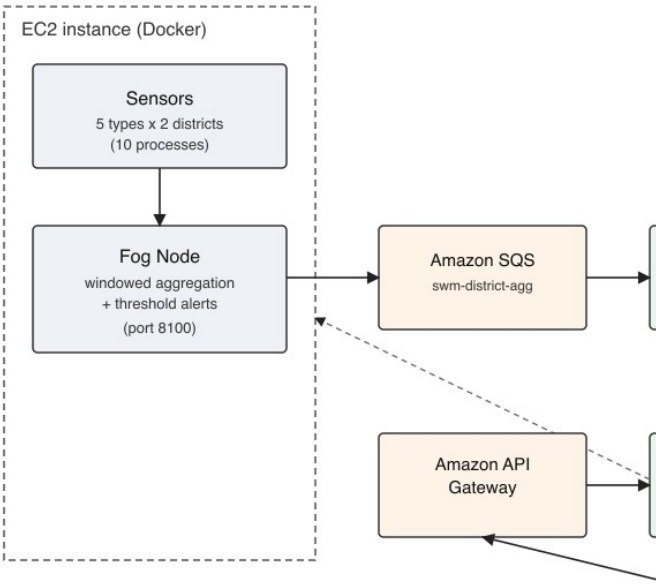

Fig. 1. Deployment topology: sensors and the fog node run on EC2; the backend (SQS, both Lambda functions, DynamoDB, API Gateway) is fully serverless; the frontend is served from S3.

E. Critical Analysis of Alternative Architectures

Several alternative designs were considered and rejected, each for a specific, statable reason rather than by default. A single monolithic EC2 instance running the fog node, the processor, and the dashboard together, with no queue at all, was rejected because it collapses the backend into exactly the single point of failure and the fixed-capacity server the brief's "scalable... queues, FaaS, autoscaling" language is written to move away from: a burst of sensor traffic would compete directly with dashboard requests for the same process's CPU and event loop, rather than being absorbed by a queue.

Amazon Kinesis Data Streams was considered in place of SQS as the fog-to-backend transport. Kinesis is the more natural choice when multiple independent consumers must each read the full stream, or when strict, per-shard ordering is required; this system has exactly one consumer (the processor Lambda) and no ordering requirement across sensor types or districts, so SQS's simpler competing-consumer queue model, its lower cost at this data volume, and its native, zero-configuration Lambda event-source-mapping integration made it the better-justified choice for this specific workload rather than a default one.

Amazon RDS (a managed relational database) was considered in place of DynamoDB. A relational schema was rejected because the access pattern here is a narrow, well-known set of key-based lookups (by sensor type and by a time-ordered range within a site) rather than ad-hoc relational queries across normalised tables, which is exactly the access pattern DynamoDB's partition-key/sort-key model is designed for; DynamoDB also removes the need to provision, patch, and pay for an always-on database instance, consistent with keeping every backend component serverless.

Finally, converting the fog node and sensors themselves to a scheduled-Lambda model, so that the entire system, not only the backend, would be serverless, was considered and explicitly deferred rather than rejected outright. A continuous sensor loop and a stateful, windowed fog buffer both map awkwardly onto Lambda's stateless, time-limited invocation model without a redesign of the buffering logic itself, and the brief's cloud-deployment requirement is explicitly scoped to the backend layer rather than the sensor and fog layer; pursuing this conversion was judged to be a larger redesign than the remaining project time justified, and is recorded as future work in Section V rather than attempted partially.

F. Security Considerations

The EC2 instance is administered exclusively through AWS Systems Manager Session Manager; no key pair was ever provisioned, and its security group has no inbound rule for port 22, so there is no SSH attack surface at all. The only inbound rule opened is for fog's own health and threshold endpoints (port 8100), which the dashboard's Lambda function needs to reach, and it exposes only read-only, non-sensitive status information rather than data or control-plane access. An earlier, now-decommissioned rule for the original EC2-hosted dashboard's port was identified as a stale, unused opening during a later verification pass and was revoked.

Both Lambda functions execute under the AWS Academy Learner Lab account's single shared LabRole, since this account's service-control policies prevent a student from creating new, more narrowly scoped IAM roles. This is a genuine limitation, not a design preference: in a production account, the processor and dashboard-API functions would each hold their own least-privilege role (the processor needing only sqs:ReceiveMessage/DeleteMessage and dynamodb:PutItem; the dashboard API needing only read-oriented DynamoDB, SQS, and Lambda-metadata actions), and this gap is recorded honestly here rather than glossed over. The dashboard's own REST API is intentionally unauthenticated, since it exposes only aggregated, non-personal environmental sensor readings for a municipal

facility, not data for which access control would be a meaningful requirement at this project's scale.

III. Implementation

A. Software Components and Libraries

The sensor, fog, processor, and dashboard components are all implemented in Node.js 20 [7], using the official AWS SDK for JavaScript v3 (@aws-sdk/client-sqs, @aws-sdk/client-dynamodb, @aws-sdk/lib-dynamodb, and @aws-sdk/client-lambda) for every AWS interaction, and Node's own built-in http module rather than a web framework such as Express, keeping the dependency surface deliberately small. The dashboard's trend visualisation is rendered client-side with Chart.js, vendored locally rather than loaded from a content-delivery network so the page has no third-party runtime dependency. Local development and continuous integration both use Docker Compose [6] together with LocalStack [8], an open-source AWS cloud emulator, so the same SQS, DynamoDB, and Lambda interactions used in production can be exercised without touching a real AWS account during day-to-day development.

Choosing Node.js consistently across all four components, rather than a different language per layer, was a deliberate simplification: it means the fake-AWS-client test-double pattern described in Section III-B, the credential-resolution logic whose bug is described in Section III-D, and the general project structure are shared idioms across the whole codebase rather than four separately maintained conventions. The trade-off accepted is that Node's single-threaded event loop is not the best fit for CPU-bound work; this project has none, since aggregation over a small, bounded per-window reading count and JSON serialisation are both cheap relative to the I/O-bound work (HTTP requests, AWS SDK calls) that dominates every component's actual running time, so the trade-off costs nothing in practice for this specific workload.

B. Testing Strategy

Every module (sensors, fog, backend processor, and backend dashboard) has its own automated test suite written with Node's built-in test runner (node:test), exercised through hand-written fake AWS client objects rather than mocking libraries, so that each unit test asserts on the exact request shape sent to a given AWS SDK call. A fake DynamoDB client, for example, is a plain object exposing only a send(command) method; a test constructs it to return a fixed set of items for a QueryCommand and then asserts that the processor's handler produced exactly the DynamoDB item shape expected, including the sort_key value, without any real network call or a general-purpose mocking framework standing between the test and the code under test.

Beyond unit tests, a load-testing script bursts a configurable number of synthetic messages directly onto the real queue and asserts, first, that the queue depth is non-zero immediately after the burst (proving the messages genuinely reached SQS rather than being silently dropped), and second, that the queue depth has either fully drained or measurably decreased within a timeout window, allowing for the underlying Lambda concurrency ceiling without treating a slower-than-expected drain as a hard failure. This two-tier assertion pattern was deliberately chosen so that the load test remains a meaningful elasticity check without becoming flaky under normal, expected throughput variation. All four modules' test suites, 113 tests in total, pass against both the LocalStack-backed development environment and, where applicable, the real AWS deployment.

C. Continuous Integration and Deployment

A GitHub Actions workflow runs the full automated test suite for all four modules on every push to the repository, then, in a second job, brings the entire stack up with Docker Compose against LocalStack and runs a dedicated end-to-end verification script that exercises the live pipeline, submits synthetic readings, and asserts that they land in DynamoDB with the expected sort_key shape, tearing the stack back down afterwards regardless of outcome so a failed run cannot leave a stale container running for the next one.

A second, separately triggered GitHub Actions workflow exists specifically to restart the EC2-hosted fog and sensor tier after an AWS Academy Learner Lab session ends and a fresh one begins, since the Learner Lab platform stops the EC2 instance between sessions; this workflow reads its AWS credentials from encrypted GitHub repository secrets rather than accepting them as plain workflow inputs, verifies that the credentials resolve to the expected AWS account before doing anything else, and only then starts the instance, waits for its Systems Manager agent to register, and re-verifies that fog is reachable both directly and through the dashboard API before reporting success.

D. Real AWS Deployment and Defects Discovered

The complete system, including the sensor and fog tier, the SQS queue, both Lambda functions, the DynamoDB table, the API Gateway endpoint, and the S3-hosted frontend, was deployed to a real AWS account (an AWS Academy Learner Lab account) in region us-east-1, satisfying the brief's requirement that the backend be deployed and tested on an actual public cloud platform rather than only a local emulator. This real deployment was directly responsible for surfacing three classes of defect that the local, LocalStack-backed test suite had not caught.

First, three separate files across the fog, processor, and dashboard components constructed their AWS SDK client credentials using a check that was true in both the local and the real-AWS case: whenever an AWS access key environment variable was present, the code applied a hardcoded, LocalStack-only static credential pair. Because real AWS Lambda and EC2 execution environments always populate that same environment variable automatically, with genuinely temporary, session-token-bearing credentials, this logic silently discarded the real temporary credentials in favour of the hardcoded pair on every real deployment, causing every DynamoDB and SQS call to fail authentication in production despite passing every LocalStack-backed test. The fix was to gate the hardcoded override on the presence of an explicit LocalStack endpoint override instead, which is the only condition that is actually unique to the local-emulator case.

Second, the dashboard's Lambda adapter initially passed through a numeric Content-Length header value copied from the original Node.js response object; AWS Lambda's HTTP response format requires every header value to be a string, and a numeric value was silently rejected, producing a generic “Internal Server Error” with no corresponding error recorded in the function's own execution logs, since the function itself completed successfully and only its response shape was rejected downstream. Third, API Gateway's own “quick create” tooling, when pointed at an existing Lambda function, did not actually attach the resource-based invoke permission it documents as automatic, which was only discovered by comparing a successful direct, authenticated Lambda

invocation against a failing request through the newly created API Gateway endpoint.

All three defects were fixed and re-verified against the live deployment: each fix was redeployed to the real Lambda functions and confirmed via a genuine, successful end-to-end invocation (a real DynamoDB write following the credential fix, a real 200 response through API Gateway following the header and permission fixes), rather than trusted from the unit test suite alone. The source code, including the fixes described above, is available at https://github.com/valletivarish/FEC_version_2, under projects/22-smart-waste-management, together with a readme.txt documenting installation, configuration, and the deployment history described in this section.

IV. Evaluation

A. Automated Test Coverage

Table II summarises the automated test suite as of the final verified deployment. All 113 tests pass against both the LocalStack-backed development environment and, for the parts of the suite that exercise real client-construction logic, the fixes verified against the real AWS deployment described in Section III-D.

TABLE II. Automated Test Suite by Module

Module	Tests	Focus
sensors	19	sampling/dispatch interval independence
fog	46	windowed aggregation, alerting, dispatch
backend/processor	11	DynamoDB item shape, sort-key logic
backend/dashboard	37	REST routes, health/priority logic

B. Live Load Test

A burst load test sent 300 synthetic messages directly to the real, deployed swm-district-agg queue in 6.15 seconds (approximately 49 messages per second), well above the sustained rate any single collection district's sensors would realistically produce. The queue depth was confirmed non-zero immediately after the burst, and on this run the real swm-processor Lambda drained the entire burst before the follow-up check even ran, evidence of real elastic throughput on the deployed system rather than a simulated result: LocalStack's own single-container Lambda executor was measurably slower at equivalent burst sizes during local development, which is the reason the load test's own drain assertion is a soft, timeout-tolerant check rather than a hard one.

C. Live Deployment Health

Polling the deployed dashboard's own /api/health endpoint twice, fifteen seconds apart, returned freshest_age_seconds of 0.136 and 6.915 respectively and items_in_table counts of 4083 and 4093, both moving as expected of a genuinely live, continuously updating pipeline rather than static or cached data. All AWS resources (the DynamoDB table, the SQS queue, both Lambda functions, the API Gateway API, and the EC2 instance) were independently confirmed healthy via direct AWS CLI calls rather than trusted from the application's own self-reported status alone.

Operationally, every backend component other than the EC2 instance runs within AWS's request-based free-tier allowances at this project's data volume; the EC2 instance itself is the only continuously billed resource, and it can be stopped between demonstration sessions without affecting the dashboard or its API, since neither depends on it being available.

D. Limitations

Three limitations of this evaluation are worth stating explicitly rather than left implicit. First, the fog and sensor tier still runs on a single EC2 instance and therefore remains a single point of failure for that tier specifically, even though the backend behind it is fully redundant AWS-managed infrastructure; the critical analysis in Section II-E explains why converting this tier to a fully serverless model was deferred rather than attempted, but the resulting availability gap is real and unresolved, not merely theoretical. Second, the AWS Academy Learner Lab account used for this deployment is itself session-based rather than persistent, so the sustained, multi-day uptime evaluation a production deployment would warrant was not possible within the account's own constraints; the evaluation in this section reflects verified behaviour within a single continuous session, not long-run reliability. Third, both Lambda functions share one broadly scoped IAM role rather than two least-privilege roles, a constraint of the Learner Lab account rather than a deliberate security design, as discussed in Section II-F; this is a genuine limitation of what could be independently verified about the deployment's security posture, not evidence that the least-privilege alternative was evaluated and rejected.

V. Conclusion

This project set out to build a complete fog and edge computing pipeline for smart waste management, satisfying the module brief's requirements for a configurable sensor and fog layer, a scalable cloud backend, and genuine deployment and testing on a public cloud platform, and to critically analyse the architectural decisions taken at each layer against the alternatives available. All three layers were implemented, tested, and deployed to a real AWS account rather than a local simulation alone.

The most significant finding of this project is not architectural but methodological: local testing against a cloud emulator, however thorough, is not a substitute for testing against the real target platform. Every defect discovered during the real AWS deployment, the credential-handling bug, the Lambda response-format bug, and the API Gateway permission gap, passed the full local test suite and a full LocalStack-backed integration test cleanly, and would have shipped undetected had the project stopped at the local-simulation stage. This directly reinforces the module's own assessment criterion that each layer be “developed, tested, and deployed” as three distinct activities, not a single combined one.

The most difficult part of this project was not writing application code but discovering the AWS Academy Learner Lab account's own platform restrictions, several of which (CloudFront being entirely blocked, and public Lambda Function URLs being blocked while API Gateway is not) are not documented anywhere and were only discoverable by attempting each service directly and reading the resulting access-denied errors. Future work would include converting the fog and sensor tier itself to a fully event-driven, scheduled-Lambda model rather than a continuously running EC2 instance, and, if the account restrictions were lifted, reintroducing a CloudFront distribution in front of the S3-hosted frontend for a custom domain and edge caching.

References

[1] Amazon Web Services, “AWS Lambda Developer Guide,” Amazon Web Services, Inc. [Online]. Available: https://docs.aws.amazon.com/lambda/ (accessed Jul. 2026).

[2] Amazon Web Services, “Amazon DynamoDB Developer Guide,” Amazon Web Services, Inc. [Online]. Available: https://docs.aws.amazon.com/dynamodb/ (accessed Jul. 2026).

[3] Amazon Web Services, “Amazon Simple Queue Service Developer Guide,” Amazon Web Services, Inc. [Online]. Available: https://docs.aws.amazon.com/sqs/ (accessed Jul. 2026).

[4] Amazon Web Services, “Amazon API Gateway Developer Guide,” Amazon Web Services, Inc. [Online]. Available: https://docs.aws.amazon.com/apigateway/ (accessed Jul. 2026).

[5] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, “Fog computing and its role in the Internet of Things,” in Proc. 1st ACM Mobile Cloud Computing Workshop (MCC ’12), Helsinki, Finland, 2012.

[6] Docker Inc., “Docker Compose overview,” Docker Documentation. [Online]. Available: https://docs.docker.com/compose/ (accessed Jul. 2026).

[7] OpenJS Foundation, “Node.js Documentation.” [Online]. Available: https://nodejs.org/docs/ (accessed Jul. 2026).

[8] LocalStack, “LocalStack Documentation.” [Online]. Available: https://docs.localstack.cloud/ (accessed Jul. 2026).

[9] P. Mell and T. Grance, “The NIST Definition of Cloud Computing,” National Institute of Standards and Technology, Gaithersburg, MD, USA, Special Publication 800-145, 2011.

[10] Amazon Web Services, “AWS Well-Architected Framework,” Amazon Web Services, Inc. [Online]. Available: https://aws.amazon.com/architecture/well-architected/ (accessed Jul. 2026).